Step 1 — ContentForge, Reachable Over HTTP for the First Time — Solution

Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 1: Project 4's multi-agent team works, but only for someone sitting at the terminal with your code and your key. This step gives it a front door with FastAPI: `POST /run-task` takes a topic as JSON and returns the team's finished article as JSON. The five agents don't change at all — the API calls them through **one** function, `run_pipeline()`, so the web layer is a thin wrapper, not a rewrite. **If not:** every caller would need Python, your code and your key installed on their own machine.

After this step the project runs as a web API: `uvicorn src.api.main:app --reload`, then `/docs` in the browser. Project 4's files (copied in during Setup) don't change.

Solution — one door into the team, typed shapes, 400/500, and tests with a faked team

Story — `pipeline.py`: this is the **one** function that invokes Project 4's team graph and turns its raw final state into plain values an API response can hold: the article text, whether it was accepted, why the run stopped, and one readable text line per routing step. **If not:** the route would reach into the graph's state dict directly — and a test couldn't swap the whole team for a fake in one single place.

```
# src/pipeline.py
# The one door into Project 4's team. run_pipeline(topic) builds and runs
# the graph and returns plain values (article, accepted, reason, steps);
# steps_from_log() turns the routing log into one text line per step.
from src.graph import build_graph
from src.state import start_state

def steps_from_log(routing_log):
    steps = []
    for entry in routing_log:
        steps.append(entry["step"] + ": " + entry["reason"])
    return steps

def run_pipeline(topic):
```

```
    graph = build_graph()
    result = graph.invoke(start_state(topic))
    last_entry = result["routing_log"][-1]
    return {
        "article": result["draft"],
        "accepted": result["accepted"],
        "reason": last_entry["reason"],
        "steps": steps_from_log(result["routing_log"]),
    }
```

Story — `api/schemas.py`: these are the API's request and response contracts, written once as Pydantic models (Doc12's `chat_schemas.py` pattern). `Field(min_length=1, max_length=200)` on the request means FastAPI rejects an empty or huge topic with a 422 automatically, *before* any model is ever called. **If not:** a 5,000-word "topic" would be sent straight to five expensive agents, and a typo in a reply field would only be found by the client parsing bad JSON.

```
# src/api/schemas.py
# The API's request/reply shapes. TaskRequest rejects an empty or huge
# topic with a 422 before the route runs; TaskResponse is the checked reply.
from pydantic import BaseModel, Field

class TaskRequest(BaseModel):
    topic: str = Field(min_length=1, max_length=200)

class TaskResponse(BaseModel):
    accepted: bool
    reason: str
    article: str
```

Story — `api/routes.py`: these routes live on an `APIRouter` (Doc12's `health_routes.py` pattern). `POST /run-task` follows Doc12's `chat_route` shape: a deliberate 400 for a blank topic, and any unexpected crash caught and turned into a plain 500 response — while `logger.exception()` still writes the real traceback to the server's own logs, just never to the client. (Also create an empty `api/__init__.py`.) **If not:** a client could see raw file paths from a traceback, and nobody on the server would know why a run actually failed.

```
# src/api/routes.py
# The API routes on an APIRouter: GET /health and POST /run-task. The POST
# route rejects a blank topic with 400 and turns any crash into a plain 500,
# keeping the real traceback on the server with logger.exception().
from fastapi import APIRouter, HTTPException

from src.api.schemas import TaskResponse
from src.pipeline import run_pipeline
from src.utils.logger import get_logger
```

```
router = APIRouter()

INTERNAL_ERROR = "Internal error. Please try again later."

@router.get("/health")
def health():
    return {"status": "ok"}

@router.post("/run-task", response_model=TaskResponse)
def run_task(request):
    logger = get_logger(__name__)
    if not request.topic.strip():
        raise HTTPException(status_code=400, detail="topic cannot be empty")

    try:
        outcome = run_pipeline(request.topic)
    except Exception:
        logger.exception("run-task failed")
        raise HTTPException(status_code=500, detail=INTERNAL_ERROR)

    logger.info(f"run-task done: {outcome['reason']}")
    return TaskResponse(
        accepted=outcome["accepted"],
        reason=outcome["reason"],
        article=outcome["article"],
    )
```

Story — `api/main.py` : this is the one file `uvicorn` actually starts. `load_config()` runs first, at import time — it reads `.env` (the agents need the key) and stops the app from starting at all if `OPENAI_API_KEY` is missing. **If not:** the server would start "fine" and then fail on every single real request, which is a far worse failure to debug.

```
# src/api/main.py
# The app uvicorn starts. load_config() runs first, so a missing
# OPENAI_API_KEY stops the server at startup, not on the first request.
from fastapi import FastAPI

from src.api.routes import router
from src.config import load_config

load_config()

app = FastAPI(title="ContentForge Pro")
app.include_router(router)
```

Story — `tests/api/test_routes.py` : this tests the API layer with pytest and `TestClient` (Doc12, Doc13's pattern), with the agent team **faked out**: `monkeypatch` swaps `routes.run_pipeline` for a function that returns a fixed answer, or one that crashes on purpose (Doc13's integration-test pattern). That means no model calls, no cost, and the exact same result every single run — true unit testing of the route logic alone. **If not:** every API test would run five real agents — slow, paid, and never quite the same twice.

```
# tests/api/test_routes.py
# The API's own behavior — status codes and replies — with Project 4's
# team FAKED: no model calls, no cost, same every time.
# Run from the repo root: pytest tests/api -v
# The app still refuses to start without OPENAI_API_KEY — any value
# works here, because no real model is ever called.
from fastapi.testclient import TestClient

from src.api import routes
from src.api.main import app

client = TestClient(app)

def fake_run_pipeline(topic):
    return {
        "article": "E-Bikes Explained\n\nUp to 100 km. [battery-review]",
        "accepted": True,
        "reason": "reviewer accepted the draft",
        "steps": [
            "research_agent: no research yet",
            "END: reviewer accepted the draft",
        ],
    }

def broken_run_pipeline(topic):
    raise RuntimeError("simulated internal failure")

def use_fake_team(monkeypatch, fake):
    monkeypatch.setattr(routes, "run_pipeline", fake)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}

def test_valid_request_is_answered(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    response = client.post("/run-task", json={"topic": "electric bikes"})
    assert response.status_code == 200
```

```
    body = response.json()
    assert body["accepted"] is True
    assert "[battery-review]" in body["article"]

def test_bad_body_is_422(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    response = client.post("/run-task", json={})
    assert response.status_code == 422

def test_blank_topic_is_400(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    response = client.post("/run-task", json={"topic": "   "})
    assert response.status_code == 400
    assert response.json() == {"detail": "topic cannot be empty"}

def test_internal_error_is_a_plain_500(monkeypatch):
    use_fake_team(monkeypatch, broken_run_pipeline)
    response = client.post("/run-task", json={"topic": "electric bikes"})
    assert response.status_code == 500
    expected = {"detail": "Internal error. Please try again later."}
    assert response.json() == expected
```

Run it (from the repo root):

```
pytest tests/api -v
uvicorn src.api.main:app --reload
```

```
tests/api/test_routes.py::test_health PASSED
tests/api/test_routes.py::test_valid_request_is_answered PASSED
tests/api/test_routes.py::test_bad_body_is_422 PASSED
tests/api/test_routes.py::test_blank_topic_is_400 PASSED
tests/api/test_routes.py::test_internal_error_is_a_plain_500 PASSED
```

Project 4's own tests stay exactly as Project 4 wrote them — plain OK/WRONG scripts you run with `python -m tests.unit.test_config` — so point pytest at `tests/api` only. New tests in this project use pytest (Doc13); Project 4's are never rewritten. Then send a request to the running server from `/docs`:

```
{"accepted":true,"reason":"reviewer accepted the draft",
 "article":"E-Bikes: How Far, How Fast ... [battery-review] ..."}
```

(Really one line — shown wrapped to fit the page. The article varies from run to run.) Try `{"topic": "deep sea mining"}` too: `"accepted":false`, `"reason":"research found no facts for this topic"` and an empty article — the team's honest stop, now visible to the client.